```python
import yfinance as yf
import pandas as pd
import numpy as np
import plotly.graph_objects as go
from plotly.subplots import make_subplots
import statsmodels.api as sm
from statsmodels.tsa.arima.model import ARIMA
from statsmodels.tsa.stattools import adfuller, acf, pacf
from scipy import stats
from scipy.stats import linregress, pearsonr, spearmanr, kendalltau,
binomtest
import warnings
import calendar
import scipy.stats as scs

warnings.filterwarnings("ignore")

# PARAMETERS & DATA DOWNLOAD
TICKER = "MSFT" # you can change here the ticker (yfinance)
START_DATE = "1999-01-01"
TEMPLATE_STYLE = "plotly_dark"

print(f"Downloading data for {TICKER} from {START_DATE} to today...\
n")
stock = yf.Ticker(TICKER)
df = stock.history(start=START_DATE)

if df.empty:
    raise ValueError("No data found for this ticker.")

if df.index.tz is not None:
    df.index = df.index.tz_localize(None)

CUTOFF_MONTH = 2
CUTOFF_DAY = 20
X_DAYS = 100
```

```
Downloading data for MSFT from 1999-01-01 to today...
```

```python
# MONTHLY RETURNS DISTRIBUTION (BOX & SCATTER)
df_box = df[['Close']].resample('ME').last()
df_box['Return'] = df_box['Close'].pct_change() * 100
df_box['Month'] = df_box.index.month
df_box['Year'] = df_box.index.year
df_box = df_box.dropna()

month_map = {i: calendar.month_abbr[i] for i in range(1, 13)}

fig_box = go.Figure()
```

```
for m in range(1, 13):
    month_data = df_box[df_box['Month'] == m]

    fig_box.add_trace(go.Box(
        y=month_data['Return'],
        name=month_map[m],
        boxpoints='all',       # Add scatter points
        jitter=0.4,            # Spread points horizontally
        pointpos=-1.8,         # Position points to the left of the
box
        marker=dict(color='#3498db', size=4, opacity=0.7),
        line=dict(color='#9b59b6', width=1.5),
        fillcolor='rgba(155, 89, 182, 0.2)',
        hovertext=month_data['Year'].astype(str),
        hovertemplate="Year: %{hovertext}<br>Return: %{y:.2f}
%<extra></extra>"
    ))

fig_box.update_layout(
    title=dict(
        text=f"<b>Monthly Returns Distribution</b><br><sup>Historical
Scatter & Box Plot Analysis — {TICKER}</sup>",
        x=0.5,
        font=dict(size=20, color='white')
    ),
    template="plotly_dark",
    paper_bgcolor='black',
    plot_bgcolor='black',
    height=600,
    margin=dict(l=50, r=50, t=100, b=50),
    showlegend=False,
    yaxis=dict(
        title="Return (%)",
        zeroline=False,
        gridcolor='rgba(255, 255, 255, 0.1)',
        tickformat=".1f",
        ticksuffix="%"
    ),
    xaxis=dict(
        title="Month",
        gridcolor='rgba(255, 255, 255, 0.05)',
        showline=True,
        linecolor='rgba(255, 255, 255, 0.2)'
    ),
    hovermode="closest"
)

# Add a dashed line at 0% for quick visual reference of profitability
fig_box.add_hline(y=0, line_dash="dash", line_color="white",
opacity=0.4)
```

```
fig_box.show()

# YTD EXCURSION  (using ARMA MODEL)
current_year = df.index[-1].year
days_passed = len(df[df.index.year == current_year])

years = df.index.year.unique()
spread_data = []

for y in years:
    df_year = df[df.index.year == y]

    if len(df_year) >= days_passed:
        df_slice = df_year.iloc[:days_passed]

        ytd_high = df_slice['High'].max()
        ytd_low = df_slice['Low'].min()
        spread_pct = ((ytd_high - ytd_low) / ytd_low) * 100
        cutoff_close = df_slice.iloc[-1]['Close']

        spread_data.append({'Year': y, 'Spread': spread_pct, 'Close':
cutoff_close})

results_df = pd.DataFrame(spread_data).dropna()
results_df = results_df[results_df['Spread'] > 0]
ts_data = results_df['Spread'].values
years_list = results_df['Year'].values

adf_result = adfuller(ts_data)
adf_p = adf_result[1]
model = ARIMA(ts_data, order=(0, 0, 1))
model_fit = model.fit()
lb_pvalue = sm.stats.acorr_ljungbox(model_fit.resid, lags=[1],
return_df=True)['lb_pvalue'].iloc[0]

colors = ['#1f3b6c'] * (len(years_list) - 1) + ['#c95c46']

fig1 = make_subplots(specs=[[{"secondary_y": True}]])
fig1.add_trace(go.Bar(
    x=years_list,
    y=ts_data,
    name="YTD Spread",
    marker_color=colors
), secondary_y=False)

fig1.add_trace(go.Scatter(
    x=years_list,
    y=results_df['Close'],
    name="Price (log)",
    line=dict(color="#53d5c5", width=2)
```

```
), secondary_y=True)

fig1.update_layout(
    title_text=f"YTD Excursion (First {days_passed} days) + Equity
(log) - {TICKER}",
    template=TEMPLATE_STYLE,
    height=500,
    bargap=0.2
)

fig1.update_yaxes(
    title_text="Spread (%)",
    secondary_y=False,
    range=[0, 60]
)

fig1.update_yaxes(
    title_text="Price (log)",
    secondary_y=True,
    showgrid=False,
    type="log",
    dtick="D1",
    tickformat=".0f"
)

fig1.show()

# MONTHLY SEASONALITY & ANNUAL CORRELATION
df_m = df[['Close']].resample('ME').last()
df_m['Ret'] = df_m['Close'].pct_change() * 100
df_m['Month'], df_m['Year'] = df_m.index.month, df_m.index.year

df_annual_ret = df['Close'].resample('YE').last().pct_change() * 100
df_annual_ret.name = 'Ann_Ret'
df_annual_ret.index = df_annual_ret.index.year
df_m = df_m.join(df_annual_ret, on='Year').dropna(subset=['Ret',
'Ann_Ret'])
month_names = [calendar.month_abbr[i] for i in range(1, 13)]

df_daily = df[['High', 'Low']].copy()
df_daily['Month'], df_daily['Year'] = df_daily.index.month,
df_daily.index.year

def calc_monthly_max_dd(group):
    if len(group) < 2: return np.nan
    cummax = group['High'].cummax()
    dd = (group['Low'] - cummax) / cummax
    return dd.min() * 100

monthly_dds = df_daily.groupby(['Year',
```

```
'Month']).apply(calc_monthly_max_dd).reset_index(name='Max_DD')
monthly_stats = []

for m in range(1, 13):
    ret_month = df_m[df_m['Month'] == m]['Ret']
    dd_month = monthly_dds[monthly_dds['Month'] == m]
['Max_DD'].dropna()
    monthly_stats.append({
        'Month': month_names[m-1],
        'Win_Rate': (ret_month > 0).mean() * 100 if len(ret_month) > 0
else 0,
        'Avg_Ret': ret_month.mean() if len(ret_month) > 0 else 0,
        'Avg_DD': dd_month.mean() if not dd_month.empty else 0,
        'Med_DD': dd_month.median() if not dd_month.empty else 0
    })

df_stats_m = pd.DataFrame(monthly_stats)

fig_bar_m = make_subplots(rows=2, cols=1, subplot_titles=(f"Positive
Month Probability ({TICKER})", "Average Intraday Max Drawdown"),
vertical_spacing=0.15)
colors_wr = ['#2ecc71' if wr >= 50 else '#e74c3c' for wr in
df_stats_m['Win_Rate']]
fig_bar_m.add_trace(go.Bar(x=df_stats_m['Month'],
y=df_stats_m['Win_Rate'], marker_color=colors_wr, name="Win Rate %"),
row=1, col=1)
fig_bar_m.add_trace(go.Bar(x=df_stats_m['Month'],
y=df_stats_m['Avg_DD'], marker_color='#c0392b', name="Avg Drawdown
(%)"), row=2, col=1)
fig_bar_m.update_layout(template=TEMPLATE_STYLE, height=600)
fig_bar_m.show()

fig_scatter_m = make_subplots(rows=3, cols=4,
subplot_titles=[f"{month_names[m-1]} vs Year" for m in range(1, 13)],
horizontal_spacing=0.05, vertical_spacing=0.1)

for m in range(1, 13):
    df_sub = df_m[df_m['Month'] == m].copy()
    if len(df_sub) > 2:
        x_val, y_val = df_sub['Ret'].values, df_sub['Ann_Ret'].values
        slope, intercept, r_value, _, _ = linregress(x_val, y_val)

        row_idx = (m-1)//4+1
        col_idx = (m-1)%4+1

        fig_scatter_m.add_trace(go.Scatter(x=x_val, y=y_val,
mode='markers', marker=dict(color='#f1c40f', size=6),
showlegend=False), row=row_idx, col=col_idx)
        fig_scatter_m.add_trace(go.Scatter(x=np.linspace(x_val.min(),
x_val.max(), 100), y=slope * np.linspace(x_val.min(), x_val.max(),
```

```
100) + intercept, mode='lines', line=dict(color='#e74c3c',
dash='dash'), showlegend=False), row=row_idx, col=col_idx)
        fig_scatter_m.add_annotation(text=f"R²: {r_value**2:.2f}",
xref=f"x{m}", yref=f"y{m}", x=0.05, y=0.95, showarrow=False,
font=dict(color="white"))

fig_scatter_m.update_layout(title_text=f"Correlation: Single Month vs
Annual Return ({TICKER})", template=TEMPLATE_STYLE, height=800)
fig_scatter_m.show()

# YTD CORRELATION & SEASONALITY ALIGNED
df_ytd = df[['Close']].copy()
df_ytd['Year'] = df_ytd.index.year

# Create sequential "Trading Day" index (1 to ~252) instead of
Calendar Day (1 to 365)
# This eliminates the sawtooth pattern caused by weekend
misalignments.
df_ytd['TradingDay'] = df_ytd.groupby('Year').cumcount() + 1

pivot_ytd = df_ytd.pivot(index='TradingDay', columns='Year',
values='Close')

# Normalize starting from the first trading day of each year
norm_ytd = ((pivot_ytd / pivot_ytd.iloc[0]) - 1) * 100

CURRENT_YEAR = norm_ytd.columns[-1]
historical_years = [y for y in norm_ytd.columns if y < CURRENT_YEAR]

# Current year series without forward filling future days
curr_series = norm_ytd[CURRENT_YEAR].dropna()
days_passed = len(curr_series)

# Calculate Correlations for historical years up to 'days_passed'
correlations_yr = []
curr_vals = curr_series.values

for y in historical_years:
    hist_series = norm_ytd[y].dropna()
    # Check if the historical year has enough days to compare
    if len(hist_series) >= days_passed and days_passed > 10:
        hist_vals = hist_series.iloc[:days_passed].values

        p, _ = pearsonr(curr_vals, hist_vals)
        s, _ = spearmanr(curr_vals, hist_vals)
        k, _ = kendalltau(curr_vals, hist_vals)

        # Average the three correlation metrics for a robust score
        if not np.isnan(p):
            correlations_yr.append({'Year': y, 'Score': (p + s + k) /
```

```
3})

# Get top 3 correlated years
correlations_yr.sort(key=lambda x: x['Score'], reverse=True)
top_3_yr = [item['Year'] for item in correlations_yr[:3]] if
correlations_yr else []

fig_seas = go.Figure()

# Add historical median
hist_data = norm_ytd[historical_years]
median_val = hist_data.median(axis=1)
fig_seas.add_trace(go.Scatter(
    x=median_val.index, y=median_val, mode='lines',
    name='Historical Median', line=dict(color="rgba(255, 255, 255,
0.4)", dash='dash', width=2)
))

# Add the top 3 correlated historical years
colors = ['#e74c3c', '#2ecc71', '#9b59b6']
for idx, year in enumerate(top_3_yr):
    score = next(item['Score'] for item in correlations_yr if
item['Year'] == year)
    fig_seas.add_trace(go.Scatter(
        x=norm_ytd[year].index, y=norm_ytd[year], mode='lines',
        name=f'{year} (Corr Score: {score:.2f})',
line=dict(color=colors[idx], width=1.5)
    ))

# Add current year (will strictly stop at 'days_passed')
fig_seas.add_trace(go.Scatter(
    x=curr_series.index, y=curr_series, mode='lines',
    name=f'{CURRENT_YEAR} (Today)', line=dict(color="#3498db",
width=4)
))

# Vertical line marking today
fig_seas.add_vline(x=days_passed, line_dash="dash",
line_color="white", annotation_text="Today")

fig_seas.update_layout(
    title_text="YTD Seasonality & Top Correlated Years (Pearson,
Spearman, Kendall)",
    template=TEMPLATE_STYLE,
    height=600,
    xaxis_title="Trading Days of the Year",
    yaxis_title="Normalized Return (%)"
)
fig_seas.show()
```

```
# NORMALIZED SEASONALITY WITH PERCENTILE BANDS
df_bands = df[['Close']].copy()
df_bands['Year'] = df_bands.index.year

# Use Trading Days to avoid weekend misalignment (the sawtooth effect)
df_bands['TradingDay'] = df_bands.groupby('Year').cumcount() + 1
pivot_bands = df_bands.pivot(index='TradingDay', columns='Year',
values='Close')

# Normalize starting from the first trading day
norm_bands = ((pivot_bands / pivot_bands.iloc[0]) - 1) * 100

CURRENT_YEAR = norm_bands.columns[-1]
PREVIOUS_YEAR = CURRENT_YEAR - 1
historical_years = [y for y in norm_bands.columns if y < CURRENT_YEAR]

# Current year strictly without forward filling the future
curr_series = norm_bands[CURRENT_YEAR].dropna()

fig_bands = go.Figure()

hist_data = norm_bands[historical_years]
median_val = hist_data.median(axis=1)
p10 = hist_data.quantile(0.10, axis=1)
p90 = hist_data.quantile(0.90, axis=1)
p25 = hist_data.quantile(0.25, axis=1)
p75 = hist_data.quantile(0.75, axis=1)

# Add all historical years as faint background lines
for y in historical_years:
    if y != PREVIOUS_YEAR:
        fig_bands.add_trace(go.Scatter(
            x=norm_bands.index, y=norm_bands[y],
            mode='lines', line=dict(color='rgba(255,255,255,0.05)',
width=1),
            showlegend=False, hoverinfo='skip'
        ))

# 10-90% Percentile Band
fig_bands.add_trace(go.Scatter(
    x=list(norm_bands.index) + list(norm_bands.index)[::-1],
    y=list(p90.values) + list(p10.values)[::-1],
    fill='toself', fillcolor='rgba(166, 107, 56, 0.4)',
    line=dict(color='rgba(255,255,255,0)'), name='10–90%'
))

# 25-75% Percentile Band
fig_bands.add_trace(go.Scatter(
    x=list(norm_bands.index) + list(norm_bands.index)[::-1],
    y=list(p75.values) + list(p25.values)[::-1],
    fill='toself', fillcolor='rgba(192, 57, 43, 0.5)',
```

```
    line=dict(color='rgba(255,255,255,0)'), name='25–75%'
))

# Historical Median
fig_bands.add_trace(go.Scatter(
    x=median_val.index, y=median_val, mode='lines',
    name='Historical Median', line=dict(color="#9b59b6", dash='dash',
width=2)
))

# Previous Year
if PREVIOUS_YEAR in norm_bands.columns:
    fig_bands.add_trace(go.Scatter(
        x=norm_bands.index, y=norm_bands[PREVIOUS_YEAR],
        mode='lines', name=str(PREVIOUS_YEAR),
line=dict(color="#e67e22", width=2)
    ))

# Current Year (stops at current day)
fig_bands.add_trace(go.Scatter(
    x=curr_series.index, y=curr_series, mode='lines',
    name=str(CURRENT_YEAR), line=dict(color="#3498db", width=3)
))

fig_bands.update_layout(
    title_text="Normalized Annual Trend with Percentile Bands",
    template=TEMPLATE_STYLE,
    height=600,
    xaxis_title="Trading Days of the Year",
    yaxis_title="Normalized Return (%)"
)
fig_bands.show()

# EXTREME DAYS & CUMULATIVE RETURNS
df_ext = df[['Close']].copy()
df_ext['Return'] = df_ext['Close'].pct_change().dropna()

top_x_dates = df_ext.nlargest(X_DAYS, 'Return').index
bottom_x_dates = df_ext.nsmallest(X_DAYS, 'Return').index

df_ext['Return_NoBest'] = np.where(df_ext.index.isin(top_x_dates), 0,
df_ext['Return'])
df_ext['Return_NoWorst'] = np.where(df_ext.index.isin(bottom_x_dates),
0, df_ext['Return'])
df_ext['Return_NoBoth'] =
np.where(df_ext.index.isin(top_x_dates.union(bottom_x_dates)), 0,
df_ext['Return'])

fig_ext = go.Figure()
fig_ext.add_trace(go.Scatter(x=df_ext.index, y=(1 +
```

```
df_ext['Return']).cumprod(), mode='lines', name='Buy & Hold',
line=dict(color='#3498db')))
fig_ext.add_trace(go.Scatter(x=df_ext.index, y=(1 +
df_ext['Return_NoBest']).cumprod(), mode='lines', name=f'W/o
Best({X_DAYS})', line=dict(color='#e67e22')))
fig_ext.add_trace(go.Scatter(x=df_ext.index, y=(1 +
df_ext['Return_NoWorst']).cumprod(), mode='lines', name=f'W/o
Worst({X_DAYS})', line=dict(color='#2ecc71')))
fig_ext.add_trace(go.Scatter(x=df_ext.index, y=(1 +
df_ext['Return_NoBoth']).cumprod(), mode='lines', name='W/o Both',
line=dict(color='#e74c3c')))
fig_ext.update_layout(title_text=f"{TICKER}: Impact of Missing
{X_DAYS} Best/Worst Days", template=TEMPLATE_STYLE, height=400,
yaxis_type="log")
fig_ext.show()

THRESHOLD_DAYS = 10
worst_to_best_count = 0
best_to_worst_count = 0

# Worst -> Best (+/- 10 giorni)
for worst_date in bottom_x_dates:
    close_best_days = [best for best in top_x_dates if abs((best -
worst_date).days) <= THRESHOLD_DAYS]
    if close_best_days:
        worst_to_best_count += 1

# Best -> Worst (+/- 10 giorni)
for best_date in top_x_dates:
    close_worst_days = [worst for worst in bottom_x_dates if
abs((worst - best_date).days) <= THRESHOLD_DAYS]
    if close_worst_days:
        best_to_worst_count += 1

worst_to_best_pct = (worst_to_best_count / X_DAYS) * 100
oltre_wb_count = X_DAYS - worst_to_best_count
oltre_wb_pct = (oltre_wb_count / X_DAYS) * 100

best_to_worst_pct = (best_to_worst_count / X_DAYS) * 100
oltre_bw_count = X_DAYS - best_to_worst_count
oltre_bw_pct = (oltre_bw_count / X_DAYS) * 100

print(f"\nTheshold ±{THRESHOLD_DAYS} days:")
print(f"  • Best->Worst within: {best_to_worst_count}/{X_DAYS}
({best_to_worst_pct:.2f}%)   beyond: {oltre_bw_count}/{X_DAYS}
({oltre_bw_pct:.2f}%)")
print(f"  • Worst->Best within: {worst_to_best_count}/{X_DAYS}
({worst_to_best_pct:.2f}%)   beyond: {oltre_wb_count}/{X_DAYS}
({oltre_wb_pct:.2f}%)")
print("-" * 50)
```

```
Theshold ±10 days:
  • Best->Worst within: 45/100 (45.00%)   beyond: 55/100 (55.00%)
  • Worst->Best within: 54/100 (54.00%)   beyond: 46/100 (46.00%)
--------------------------------------------------
```

```
# DISTANCE FROM SMA 200
df_sma = df[['Close']].copy()
df_sma['SMA_200'] = df_sma['Close'].rolling(window=200).mean()
df_sma['Dist_SMA_200'] = ((df_sma['Close'] - df_sma['SMA_200']) /
df_sma['SMA_200']) * 100
df_sma = df_sma.dropna()

fig_sma = go.Figure()
fig_sma.add_trace(go.Scatter(x=df_sma.index, y=df_sma['Dist_SMA_200'],
mode='lines', line=dict(color='#3498db'), name='Dist %'))
fig_sma.add_hline(y=5, line_dash="dash", line_color="#2ecc71")
fig_sma.add_hline(y=-5, line_dash="dash", line_color="#2ecc71")
fig_sma.update_layout(title_text=f"{TICKER} - % Distance from SMA
200", template=TEMPLATE_STYLE, height=450)
fig_sma.show()

# BEAR MARKET & SECOND DRAWDOWN ANALYSIS
df_bm = df[['Close']].copy()
df_bm['CumMax'] = df_bm['Close'].cummax()
df_bm['Drawdown'] = (df_bm['Close'] - df_bm['CumMax']) /
df_bm['CumMax']

bear_episodes = []
i = 0

while i < len(df_bm):
    if df_bm['Drawdown'].iloc[i] <= -0.20:
        peak_date = df_bm['Close'].iloc[:i+1].idxmax()
        peak_price = df_bm['Close'].loc[peak_date]
        rec_mask = df_bm['Close'].iloc[i:] >= peak_price
        end_idx = i + rec_mask.argmax() if rec_mask.any() else
len(df_bm) - 1
        bear_slice =
df_bm['Close'].loc[peak_date:df_bm.index[end_idx]]
        bear_min_date = bear_slice.idxmin()

        sec_dds = []
        rec_phase =
df_bm['Close'].loc[bear_min_date:df_bm.index[end_idx]]

        if len(rec_phase) > 1:
            r_cummax = rec_phase.cummax()
            r_dd = (rec_phase - r_cummax) / r_cummax
            j = 0
```

```
            while j < len(rec_phase):
                if r_dd.iloc[j] <= -0.10:
                    l_peak = r_cummax.iloc[j]
                    l_peak_date = rec_phase.iloc[:j+1]
[rec_phase.iloc[:j+1] == l_peak].index[-1]
                    l_rec_mask = rec_phase.iloc[j:] >= l_peak
                    l_rec_idx = j + l_rec_mask.argmax() if
l_rec_mask.any() else len(rec_phase) - 1
                    l_bottom =
rec_phase.loc[l_peak_date:rec_phase.index[l_rec_idx]].idxmin()
                    sec_dds.append({'Start': l_peak_date, 'Bottom':
l_bottom})
                    j = l_rec_idx + 1
                else:
                    j += 1

        bear_episodes.append({'Peak': peak_date, 'Min': bear_min_date,
'Secs': sec_dds})
        i = end_idx + 1
    else:
        i += 1

fig_bm = go.Figure(go.Scatter(x=df_bm.index, y=df_bm['Close'],
mode='lines', line=dict(color='white'), name='Close'))
for ep in bear_episodes:
    fig_bm.add_vrect(x0=ep['Peak'], x1=ep['Min'], fillcolor="#e74c3c",
opacity=0.3, layer="below")
    for sec in ep['Secs']:
        fig_bm.add_vrect(x0=sec['Start'], x1=sec['Bottom'],
fillcolor="#f1c40f", opacity=0.4, layer="below")

fig_bm.update_layout(title="Bear Markets (Red >= -20%) and Aftershocks
(Gold >= -10%)", template=TEMPLATE_STYLE, yaxis_type="log",
height=500)
fig_bm.show()

# 10. BUY THE DIP STRATEGY DASHBOARD
df_btd = df[['Close']].copy()
df_btd['Returns'] = df_btd['Close'].pct_change()
df_btd['Signal'] = (df_btd['Returns'].shift(1) < 0).astype(int)
df_btd['Strategy_Ret'] = df_btd['Returns'] * df_btd['Signal']

fig_equity = go.Figure()
fig_equity.add_trace(go.Scatter(x=df_btd.index, y=(1 +
df_btd['Returns'].fillna(0)).cumprod(), name='Buy & Hold',
line=dict(color='rgba(255, 255, 255, 0.3)', width=1)))
fig_equity.add_trace(go.Scatter(x=df_btd.index, y=(1 +
df_btd['Strategy_Ret'].fillna(0)).cumprod(), name='Buy the Dip',
line=dict(color='#e74c3c', width=2)))
fig_equity.update_layout(title="Performance: Buy the Dip vs Buy &
```

```
Hold", template=TEMPLATE_STYLE, yaxis_type="log", height=400)
fig_equity.show()

# VOLATILITY COMPRESSION
df_vol = df[['Close']].copy()
df_vol['Ret_Monthly'] = df_vol['Close'].pct_change(21) * 100
df_vol['Vol_30d'] =
df_vol['Close'].pct_change().rolling(window=30).std() * np.sqrt(252) *
100

fig_comp = make_subplots(rows=2, cols=1, shared_xaxes=True,
subplot_titles=("Monthly Returns Compression", "Annualized Volatility
(30d)"))
fig_comp.add_trace(go.Scatter(x=df_vol.index, y=df_vol['Ret_Monthly'],
fill='tozeroy', line=dict(color='#f1c40f', width=1.5), name='Monthly
Ret'), row=1, col=1)
fig_comp.add_trace(go.Scatter(x=df_vol.index, y=df_vol['Vol_30d'],
line=dict(color='#e74c3c', width=2), name='Vol 30d'), row=2, col=1)
fig_comp.update_layout(title=f"Volatility Compression: {TICKER}",
template=TEMPLATE_STYLE, height=600, showlegend=False)
fig_comp.show()

# MOMENTUM VS MEAN REVERSION
ROLL_WIN = 252
df_mr = df[['Close']].copy()
df_mr['Ret'] = df_mr['Close'].pct_change()
df_mr['Sign'] = np.where(df_mr['Ret'] > 0, 1, -1)
df_mr['Next_Sign'] = df_mr['Sign'].shift(-1)

up_cond = df_mr['Sign'] == 1
down_cond = df_mr['Sign'] == -1
is_up_next = df_mr['Next_Sign'] == 1

df_mr['P_Up_Up'] = (is_up_next &
up_cond).rolling(window=ROLL_WIN).sum() /
up_cond.rolling(window=ROLL_WIN).sum().replace(0, np.nan)
df_mr['P_Up_Down'] = (is_up_next &
down_cond).rolling(window=ROLL_WIN).sum() /
down_cond.rolling(window=ROLL_WIN).sum().replace(0, np.nan)
df_mr = df_mr.dropna(subset=['P_Up_Up', 'P_Up_Down'])
df_mr['MR_Dominance'] = df_mr['P_Up_Down'] - df_mr['P_Up_Up']

fig_mr = make_subplots(rows=2, cols=1, shared_xaxes=True,
row_heights=[0.7, 0.3], vertical_spacing=0.08,
subplot_titles=("Conditional Probabilities", "Relative Dominance"))
fig_mr.add_trace(go.Scatter(x=df_mr.index, y=df_mr['P_Up_Up'] * 100,
mode='lines', name='Momentum', line=dict(color='#ff007f', width=2)),
row=1, col=1)
fig_mr.add_trace(go.Scatter(x=df_mr.index, y=df_mr['P_Up_Down'] * 100,
mode='lines', name='Mean Reversion', line=dict(color='#00f2ff',
```

```
width=2)), row=1, col=1)
fig_mr.add_trace(go.Bar(x=df_mr.index, y=df_mr['MR_Dominance'] * 100,
marker_color=['#00f2ff' if v > 0 else '#ff007f' for v in
df_mr['MR_Dominance']], name='MR Dominance'), row=2, col=1)
fig_mr.update_layout(title=f"{TICKER} — Momentum vs Mean Reversion",
template=TEMPLATE_STYLE, height=800)
fig_mr.show()

# ANNUAL RETURNS, BASE 100, AND Z-SCORE
df_yr = df['Close'].resample('YE').last()
df_yr_returns = df_yr.pct_change().dropna() * 100

years = df_yr_returns.index.year
returns_vals = df_yr_returns.values

# Calculate Base 100 Equity Line
base_100 = 100 * (1 + df_yr_returns / 100).cumprod()

# Calculate 5-Year Moving Average of returns
ma_5_returns = df_yr_returns.rolling(window=5).mean()

# Calculate Z-Score
mean_ret = df_yr_returns.mean()
std_ret = df_yr_returns.std()
z_scores = (df_yr_returns - mean_ret) / std_ret

# Calculate Hit Ratio and historical CAGR
hit_ratio = (df_yr_returns > 0).mean() * 100
total_years = (df.index[-1] - df.index[0]).days / 365.25
cagr = ((df['Close'].iloc[-1] / df['Close'].iloc[0]) ** (1 /
total_years) - 1) * 100

# Create Subplots
fig_z = make_subplots(
    rows=2, cols=1,
    shared_xaxes=True,
    vertical_spacing=0.1,
    row_heights=[0.5, 0.5],
    specs=[[{"secondary_y": True}], [{"secondary_y": False}]]
)

# --- ROW 1: Annual Returns, MA 5, and Base 100 ---
colors_ret = ['#2ecc71' if val >= 0 else '#e74c3c' for val in
returns_vals]

fig_z.add_trace(go.Bar(
    x=years, y=returns_vals,
    marker_color=colors_ret, name="Annual Return (%)",
    opacity=0.8
), row=1, col=1, secondary_y=False)
```

```
fig_z.add_trace(go.Scatter(
    x=years, y=ma_5_returns,
    mode='lines', name="5-Year MA",
    line=dict(color="#f1c40f", width=2, dash='dot')
), row=1, col=1, secondary_y=False)

fig_z.add_trace(go.Scatter(
    x=years, y=base_100,
    mode='lines', name="Capital (Base 100)",
    line=dict(color="#3498db", width=3)
), row=1, col=1, secondary_y=True)

# --- ROW 2: Z-Score ---
fig_z.add_trace(go.Scatter(
    x=years, y=z_scores,
    mode='lines+markers', name="Z-Score",
    line=dict(color="#d8973c", width=2),
    marker=dict(size=8, color="#d8973c")
), row=2, col=1)

# Z-Score Background Zones
fig_z.add_hrect(
    y0=-1, y1=1, line_width=0,
    fillcolor="white", opacity=0.1,
    row=2, col=1
)

fig_z.add_hline(y=0, line_dash="dash", line_color="white",
opacity=0.7, row=2, col=1)
fig_z.add_hline(y=2, line_dash="dot", line_color="gray", opacity=0.7,
row=2, col=1)
fig_z.add_hline(y=-2, line_dash="dot", line_color="gray", opacity=0.7,
row=2, col=1)

# Annotations for Hit Ratio and CAGR
fig_z.add_annotation(
    text=f"Hit Ratio: {hit_ratio:.1f}%   •   CAGR (historical):
{cagr:.2f}%",
    xref="paper", yref="paper",
    x=0.0, y=1.05,
    showarrow=False,
    font=dict(size=14, color="white"),
    xanchor="left",
    yanchor="bottom"
)

# Layout adjustments
fig_z.update_layout(
    title_text=f"Returns Overview & Z-Score — {TICKER}",
```

```
    template=TEMPLATE_STYLE,
    height=800,
    showlegend=True,
    legend=dict(orientation="h", yanchor="bottom", y=1.02,
xanchor="right", x=1)
)

fig_z.update_yaxes(title_text="Return (%)", row=1, col=1,
secondary_y=False)
fig_z.update_yaxes(title_text="Base 100 (log)", type="log", row=1,
col=1, secondary_y=True)
fig_z.update_yaxes(title_text="Z-Score", row=2, col=1)
fig_z.update_xaxes(title_text="Year", row=2, col=1)

fig_z.show()

# Adjusted from "Ergodicity" concept to Regime Shift Analysis

WINDOW_ERG = 252       # 252 trading days (approx 1 year for stocks)
THRESHOLD_SIGMA = 2.0  # Threshold multiplier

df_erg = df[['Close']].copy()

# 1. Log Returns: ln(Pt / Pt-1)
df_erg['Log_Return'] = np.log(df_erg['Close'] /
df_erg['Close'].shift(1))

# 2. Expanding Mean (Long-term Memory) & Rolling Mean (Short-term
behavior)
df_erg['Expanding_Mean'] = df_erg['Log_Return'].expanding().mean()
df_erg['Rolling_Mean'] =
df_erg['Log_Return'].rolling(window=WINDOW_ERG).mean()

# 3. Difference (Spread)
df_erg['Diff'] = df_erg['Rolling_Mean'] - df_erg['Expanding_Mean']
df_erg = df_erg.dropna()

# 4. Statistics
mu_diff = df_erg['Diff'].mean()
sigma_diff = df_erg['Diff'].std()

upper_thresh = mu_diff + (THRESHOLD_SIGMA * sigma_diff)
lower_thresh = mu_diff - (THRESHOLD_SIGMA * sigma_diff)

# Identify Regime Breaks ("Non-Ergodic" moments)
df_erg['Regime_Break'] = (df_erg['Diff'] > upper_thresh) |
(df_erg['Diff'] < lower_thresh)

current_state = "REGIME BREAK" if df_erg['Regime_Break'].iloc[-1] else
"MEAN REVERTING"
state_color = "#e74c3c" if df_erg['Regime_Break'].iloc[-1] else
```

```
"#2ecc71"


fig_erg = make_subplots(
    rows=3, cols=1,
    shared_xaxes=False,
    vertical_spacing=0.1,
    row_heights=[0.5, 0.3, 0.2],
    subplot_titles=(
        f"{TICKER} Price (Log) | Current State: <span style='color:
{state_color}'>{current_state}</span>",
        "Short-term vs Long-term Mean Returns",
        "Distribution of Difference (Rolling - Expanding)"
    )
)

# Row 1: Price Chart (Log Scale)
fig_erg.add_trace(go.Scatter(
    x=df_erg.index, y=df_erg['Close'],
    mode='lines', line=dict(color='rgba(255, 255, 255, 0.7)',
width=1.5), name='Price'
), row=1, col=1)

# Highlight breaks with markers
break_events = df_erg[df_erg['Regime_Break']]
fig_erg.add_trace(go.Scatter(
    x=break_events.index, y=break_events['Close'],
    mode='markers', marker=dict(color='#e74c3c', size=5), name='Regime
Breaks'
), row=1, col=1)

# Row 2: Means Comparison
fig_erg.add_trace(go.Scatter(
    x=df_erg.index, y=df_erg['Expanding_Mean'],
    mode='lines', line=dict(color='#f39c12', width=1.5, dash='dot'),
name='Expanding Mean'
), row=2, col=1)

fig_erg.add_trace(go.Scatter(
    x=df_erg.index, y=df_erg['Rolling_Mean'],
    mode='lines', line=dict(color='#00cec9', width=1.5), name='Rolling
Mean'
), row=2, col=1)

# "Allowed" Band (Ergodic Zone)
fig_erg.add_trace(go.Scatter(
    x=list(df_erg.index) + list(df_erg.index)[::-1],
    y=list(df_erg['Expanding_Mean'] + THRESHOLD_SIGMA * sigma_diff) +
list(df_erg['Expanding_Mean'] - THRESHOLD_SIGMA * sigma_diff)[::-1],
    fill='toself', fillcolor='rgba(46, 204, 113, 0.1)',
```

```
    line=dict(color='rgba(255,255,255,0)'), name='Mean Reverting
Zone', hoverinfo='skip'
), row=2, col=1)

# Row 3: Histogram of Difference
fig_erg.add_trace(go.Histogram(
    x=df_erg['Diff'], nbinsx=80,
    marker_color='#3498db', name='Diff Distribution'
), row=3, col=1)

# Threshold lines for Histogram
fig_erg.add_vline(x=upper_thresh, line_dash="dash",
line_color="#e74c3c", annotation_text=f"+{THRESHOLD_SIGMA}σ", row=3,
col=1)
fig_erg.add_vline(x=lower_thresh, line_dash="dash",
line_color="#e74c3c", annotation_text=f"-{THRESHOLD_SIGMA}σ", row=3,
col=1)

# Layout adjustments
fig_erg.update_layout(
    title_text=f"<b>Stationarity & Regime Shift
Dashboard</b><br><sup>{TICKER} - Rolling vs Expanding Memory
Breaks</sup>",
    template=TEMPLATE_STYLE,
    height=900,
    hovermode="x unified",
    showlegend=True
)
fig_erg.update_yaxes(type="log", title_text="Price (log)", row=1,
col=1)
fig_erg.update_yaxes(title_text="Log Return", row=2, col=1)
fig_erg.update_yaxes(title_text="Count", row=3, col=1)
fig_erg.update_xaxes(title_text="Difference", row=3, col=1)

fig_erg.show()

# ROLLING 52-WEEK HIGHS & FORWARD RETURNS

df_roll = df.copy()
df_roll['Year'] = df_roll.index.year
df_roll['Month'] = df_roll.index.month

# 1. Identify 52-week (252 trading days) rolling highs
df_roll['Rolling_High'] = df_roll['High'].rolling(window=252,
min_periods=50).max()
is_new_high = df_roll['High'] >= df_roll['Rolling_High']
rolling_high_dates = df_roll[is_new_high].index

# 2. Calculate forward returns
df_roll['Fwd_1W'] = df_roll['Close'].shift(-5) / df_roll['Close'] - 1
```

```
df_roll['Fwd_1M'] = df_roll['Close'].shift(-21) / df_roll['Close'] - 1
df_roll['Fwd_3M'] = df_roll['Close'].shift(-63) / df_roll['Close'] - 1
df_roll['Fwd_6M'] = df_roll['Close'].shift(-126) / df_roll['Close'] -
1
df_roll['Fwd_1Y'] = df_roll['Close'].shift(-252) / df_roll['Close'] -
1

highs_data = df_roll.loc[rolling_high_dates].copy()

# 3. Count occurrences by month
monthly_counts = highs_data['Month'].value_counts().reindex(range(1,
13), fill_value=0)
month_names_short = [calendar.month_abbr[i] for i in range(1, 13)]

# 4. Plotting setup
fig_rh = make_subplots(
    rows=3, cols=5,
    specs=[
        [{"colspan": 5}, None, None, None, None],
        [{"colspan": 5}, None, None, None, None],
        [{}, {}, {}, {}, {}]
    ],
    subplot_titles=(
        f"Equity Line (Log) & 52-Week Highs - {TICKER}",
        "Monthly Distribution of 52-Week Highs",
        "Fwd 1W (~5d)", "Fwd 1M (~21d)", "Fwd 3M (~63d)", "Fwd 6M
(~126d)", "Fwd 1Y (~252d)"
    ),
    vertical_spacing=0.12
)

# Row 1: Equity & New Highs
fig_rh.add_trace(go.Scatter(
    x=df.index, y=df['Close'],
    mode='lines', name='Close',
    line=dict(color='#3498db', width=1.5)
), row=1, col=1)

fig_rh.add_trace(go.Scatter(
    x=highs_data.index, y=highs_data['Close'],
    mode='markers', name='52-Week High',
    marker=dict(color='#e74c3c', size=6, opacity=0.6,
line=dict(color='white', width=0.5)),
    hovertemplate="Date: %{x}<br>Close: %{y:.2f}<extra></extra>"
), row=1, col=1)

# Row 2: Monthly Bar Chart
fig_rh.add_trace(go.Bar(
    x=month_names_short, y=monthly_counts.values,
    marker_color='#9b59b6', name='Month Count',
```

```
    hovertemplate="Month: %{x}<br>Count: %{y}<extra></extra>"
), row=2, col=1)

# Row 3: Forward Returns Histograms
horizons = [('Fwd_1W', 1), ('Fwd_1M', 2), ('Fwd_3M', 3), ('Fwd_6M',
4), ('Fwd_1Y', 5)]
colors_hist = ['#1abc9c', '#2ecc71', '#f1c40f', '#e67e22', '#c0392b']

for (col_name, col_idx), color in zip(horizons, colors_hist):
    valid_data = highs_data[col_name].dropna() * 100

    fig_rh.add_trace(go.Histogram(
        x=valid_data,
        nbinsx=15,
        marker_color=color,
        name=col_name,
        opacity=0.8
    ), row=3, col=col_idx)

    if not valid_data.empty:
        mean_val = valid_data.mean()
        median_val = valid_data.median()

        fig_rh.add_vline(x=mean_val, line_width=2, line_dash="solid",
line_color="white", row=3, col=col_idx)
        fig_rh.add_vline(x=median_val, line_width=2, line_dash="dot",
line_color="black", row=3, col=col_idx)

# Layout configuration
fig_rh.update_layout(
    template=TEMPLATE_STYLE,
    height=1000,
    title_text=f"<b>Rolling 52-Week Highs & Forward
Returns</b><br><sup>{TICKER}</sup>",
    showlegend=False,
    bargap=0.1
)

fig_rh.update_yaxes(type='log', row=1, col=1)

# Zero-line for histograms
for i in range(1, 6):
    fig_rh.add_vline(x=0, line_width=1, line_dash="dash",
line_color="gray", row=3, col=i)

fig_rh.show()

# MARKET ENTROPY DASHBOARD

df_ent = df.copy()
```

```
df_ent['Ret'] = df_ent['Close'].pct_change().dropna()

WINDOW = 63

def calc_shannon(x):
    counts, _ = np.histogram(x, bins=10)
    return scs.entropy(counts)

df_ent['Shannon_Ret'] =
df_ent['Ret'].rolling(window=WINDOW).apply(calc_shannon, raw=True)
df_ent['Vol_Norm'] = df_ent['Ret'].rolling(window=WINDOW).std()
df_ent['Skew_Norm'] =
df_ent['Ret'].rolling(window=WINDOW).skew().abs()

for col in ['Shannon_Ret', 'Vol_Norm', 'Skew_Norm']:
    _min, _max = df_ent[col].min(), df_ent[col].max()
    df_ent[f'{col}_01'] = (df_ent[col] - _min) / (_max - _min)

def calc_permutation_entropy(x):
    if len(x) < 3: return np.nan
    triplets = np.vstack([x[:-2], x[1:-1], x[2:]]).T
    patterns = np.apply_along_axis(lambda arr: tuple(np.argsort(arr)),
1, triplets)
    unique_patterns, counts = np.unique(patterns, axis=0,
return_counts=True)
    probs = counts / len(triplets)
    return -np.sum(probs * np.log2(probs)) / np.log2(6)

df_ent['Perm_Ent'] =
df_ent['Close'].rolling(window=WINDOW).apply(calc_permutation_entropy,
raw=True)

FWD_WINDOW = 21
df_ent['Fwd_Ret'] = df_ent['Close'].shift(-FWD_WINDOW) /
df_ent['Close'] - 1

df_ent = df_ent.dropna()

p33_perm = df_ent['Perm_Ent'].quantile(0.33)
p67_perm = df_ent['Perm_Ent'].quantile(0.67)

df_ent['Regime'] = pd.cut(
    df_ent['Perm_Ent'],
    bins=[-np.inf, p33_perm, p67_perm, np.inf],
    labels=['Low (Ordered)', 'Neutral', 'High (Chaotic)']
)

regime_fwd = df_ent.groupby('Regime')['Fwd_Ret'].mean() * 100
df_ent['Perm_Pctile'] =
df_ent['Perm_Ent'].rolling(window=252).apply(lambda x:
```

```
scs.percentileofscore(x, x[-1]))

# --- 3. PLOTTING (4x1 Grid) ---
fig_ent = make_subplots(
    rows=4, cols=1,
    specs=[
        [{"secondary_y": True}],
        [{}],
        [{}],
        [{}]
    ],
    subplot_titles=(
        "1: Shannon, Vol, Skew vs Price",
        "2: Permutation Entropy (m=3)",
        "3: Avg Fwd Returns by Entropy Regime",
        "4: Historical Percentile (Rolling 1Y)"
    ),
    vertical_spacing=0.08
)

# Row 1
fig_ent.add_trace(go.Scatter(x=df_ent.index, y=df_ent['Close'],
name="Price", line=dict(color='white')), row=1, col=1,
secondary_y=True)
fig_ent.add_trace(go.Scatter(x=df_ent.index,
y=df_ent['Shannon_Ret_01'], name="Shannon", line=dict(color='#3498db',
width=1)), row=1, col=1, secondary_y=False)
fig_ent.add_trace(go.Scatter(x=df_ent.index, y=df_ent['Vol_Norm_01'],
name="Vol", line=dict(color='#e74c3c', width=1)), row=1, col=1,
secondary_y=False)
fig_ent.add_trace(go.Scatter(x=df_ent.index, y=df_ent['Skew_Norm_01'],
name="Skew", line=dict(color='#2ecc71', width=1)), row=1, col=1,
secondary_y=False)

# Row 2
fig_ent.add_trace(go.Scatter(x=df_ent.index, y=df_ent['Perm_Ent'],
name="Perm. Entropy", line=dict(color='#f1c40f')), row=2, col=1)
fig_ent.add_hline(y=p33_perm, line_dash="dash", line_color="#2ecc71",
annotation_text="P33 (Order)", row=2, col=1)

# Row 3
colors_regime = ['#2ecc71', '#95a5a6', '#e74c3c']
fig_ent.add_trace(go.Bar(x=regime_fwd.index, y=regime_fwd.values,
marker_color=colors_regime, name="Avg Fwd Ret %"), row=3, col=1)

# Row 4
fig_ent.add_trace(go.Scatter(x=df_ent.index, y=df_ent['Perm_Pctile'],
name="Entropy Pctile", line=dict(color='#9b59b6')), row=4, col=1)
fig_ent.add_hrect(y0=80, y1=100, fillcolor="#e74c3c", opacity=0.2,
line_width=0, row=4, col=1)
```

```
fig_ent.add_hrect(y0=0, y1=20, fillcolor="#2ecc71", opacity=0.2,
line_width=0, row=4, col=1)

fig_ent.update_layout(
    height=1200,
    template=TEMPLATE_STYLE,
    title_text=f"<b>Market Entropy
Dashboard</b><br><sup>{TICKER}</sup>",
    showlegend=False
)
fig_ent.update_yaxes(type="log", row=1, col=1, secondary_y=True)
fig_ent.show()

# DYNAMIC POSITION SIZING (FRACTIONAL KELLY CRITERION)

df_k = df[['Close']].copy()

# Base strategy: Buy after a down day, hold for 5 days
df_k['Daily_Ret'] = df_k['Close'].pct_change()
df_k['Signal'] = (df_k['Daily_Ret'].shift(1) < 0).astype(int)
df_k['Fwd_5d_Ret'] = df_k['Close'].shift(-5) / df_k['Close'] - 1

trades = df_k[df_k['Signal'] == 1]['Fwd_5d_Ret'].dropna()

win_rate = (trades > 0).mean()
loss_rate = 1 - win_rate

avg_win = trades[trades > 0].mean()
avg_loss = abs(trades[trades < 0].mean())
payoff_ratio = avg_win / avg_loss if avg_loss != 0 else 0

# Kelly Formula
kelly_pct = win_rate - (loss_rate / payoff_ratio) if payoff_ratio > 0
else 0
kelly_pct = max(0, kelly_pct) * 100

half_kelly = kelly_pct / 2
quarter_kelly = kelly_pct / 4

# Plotting
fig_k = go.Figure()

fig_k.add_trace(go.Indicator(
    mode="number+gauge",
    value=half_kelly,
    domain={'x': [0.3, 1], 'y': [0.1, 0.9]},
    title={'text': f"<b>Half-Kelly Optimal Risk (%)</b><br><span
style='color: gray; font-size:0.8em'>Win Rate: {win_rate*100:.1f}% |
Payoff: {payoff_ratio:.2f}</span>", 'font': {"size": 18, "color":
"white"}},
```

```
    gauge={
        'axis': {'range': [0, max(20, kelly_pct + 5)], 'tickwidth': 1,
'tickcolor': "white"},
        'bar': {'color': "#3498db"},
        'bgcolor': "rgba(0,0,0,0)",
        'borderwidth': 2,
        'bordercolor': "gray",
        'steps': [
            {'range': [0, quarter_kelly], 'color': "rgba(46, 204, 113,
0.4)"},
            {'range': [quarter_kelly, half_kelly], 'color': "rgba(241,
196, 15, 0.4)"},
            {'range': [half_kelly, kelly_pct], 'color': "rgba(231, 76,
60, 0.4)"}
        ],
        'threshold': {
            'line': {'color': "red", 'width': 4},
            'thickness': 0.75,
            'value': kelly_pct
        }
    }
))

fig_k.update_layout(
    template=TEMPLATE_STYLE,
    height=350,
    margin=dict(l=20, r=20, t=50, b=20),
    title_text=f"Dynamic Position Sizing Calculator - {TICKER}"
)

fig_k.add_annotation(x=0, y=0.8, text=f"<b>Trade Stats (5d Hold)</b>",
showarrow=False, font=dict(color="white", size=14), xanchor="left")
fig_k.add_annotation(x=0, y=0.6, text=f"Total Setup Occurrences:
{len(trades)}", showarrow=False, font=dict(color="gray", size=13),
xanchor="left")
fig_k.add_annotation(x=0, y=0.4, text=f"Full Kelly (Max Risk):
{kelly_pct:.1f}%", showarrow=False, font=dict(color="#e74c3c",
size=13), xanchor="left")
fig_k.add_annotation(x=0, y=0.25, text=f"Half Kelly (Suggested):
{half_kelly:.1f}%", showarrow=False, font=dict(color="#f1c40f",
size=13), xanchor="left")
fig_k.add_annotation(x=0, y=0.1, text=f"Quarter Kelly (Safe):
{quarter_kelly:.1f}%", showarrow=False, font=dict(color="#2ecc71",
size=13), xanchor="left")

fig_k.show()
```